

# Backward Lens: Projecting Language Model Gradients into the Vocabulary Space

Shahar Katz<sup>1,2</sup> Yonatan Belinkov<sup>1</sup> Mor Geva<sup>2</sup> Lior Wolf<sup>2</sup>

<sup>1</sup>Faculty of Computer Science, Technion – Israel Institute of Technology

<sup>2</sup>Blavatnik School of Computer Science, Tel Aviv University

{shaharkatz3@mail, morgeva@tauex, wolf@cs}.tau.ac.il, belinkov@technion.ac.il

## Abstract

Understanding how Transformer-based Language Models (LMs) learn and recall information is a key goal of the deep learning community. Recent interpretability methods project weights and hidden states obtained from the forward pass to the models’ vocabularies, helping to uncover how information flows within LMs. In this work, we extend this methodology to LMs’ backward pass and gradients. We first prove that a gradient matrix can be cast as a low-rank linear combination of its forward and backward passes’ inputs. We then develop methods to project these gradients into vocabulary items and explore the mechanics of how new information is stored in the LMs’ neurons. Our code is available at: <https://github.com/shacharKZ/BackwardLens>.

## 1 Introduction

Deep learning models consist of layers, which are parameterized by matrices that are trained using a method known as backpropagation. This process involves the creation of gradient matrices that are used to update the models’ layers. Backpropagation has been playing a major role in interpreting deep learning models and multiple lines of study aggregate the gradients to provide explainability (Simonyan et al., 2014; Sanyal and Ren, 2021; Chefer et al., 2022; Sarti et al., 2023; Miglani et al., 2023).

Recent interpretability works have introduced methods to project the weights and intermediate activations of Transformer-based LMs (Vaswani et al., 2017) into the vocabulary space. The seminal “Logit Lens” method (nostalgebraist, 2020) has paved the way to explaining LMs’ behavior during inference (Geva et al., 2022a; Dar et al., 2022; Ram et al., 2023), including directly interpreting individual neurons (Geva et al., 2021; Katz and Belinkov, 2023). Our work is the first, as far as we can ascertain, to project LM *gradients* to the vocabulary space. Furthermore, modern LMs contain

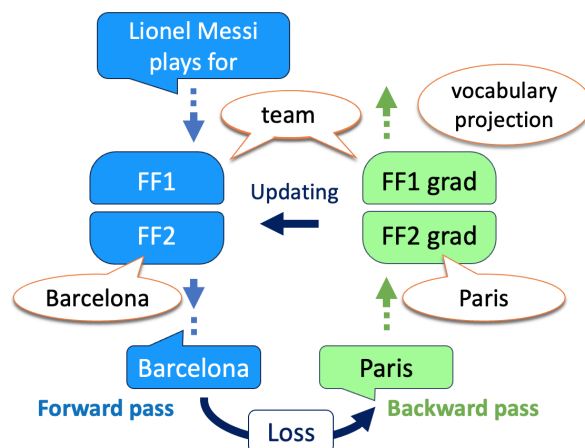


Figure 1: An illustration depicting the tokens promoted by a single LM’s MLP layer and its gradient during the forward and backward pass when editing the model to answer “Paris” for the prompt “Lionel Messi plays for”. The gradients (in green) of the first MLP matrix,  $FF_1$ , attempt to imprint into the model’s weight (in blue) the information that  $FF_1$  encountered during the forward pass. Utilizing a vocabulary projection method, we reveal that this information represents the token “team”. The gradients of the second MLP matrix,  $FF_2$ , aim to shift the information encoded within  $FF_2$  towards the embedding of the new target.

thousands of neurons in each layer, while certain features are likely distributed across multiple neurons (Elhage et al., 2022; Cunningham et al., 2023). These issues are handled in our examination of the gradient matrices by performing a decomposition of provably low-rank matrices.

Despite the popularity of LMs, our understanding of their behavior remains incomplete (Bender et al., 2021; Dwivedi et al., 2023), particularly regarding how LMs acquire new knowledge during training and the mechanisms by which they store and recall it (Dai et al., 2022; Geva et al., 2021, 2023; Meng et al., 2023). We identify a mechanism we refer to as “imprint and shift”, which captures how information is stored in the feed-forward

(MLP) module of the transformer layer. This module has two fully connected layers,  $FF_1$  and  $FF_2$ . The “imprint” refers to the first layer, to or from which the learning process adds or subtracts copies of the intermediate inputs encountered during the forward pass. The “shift” refers to the second matrix, where the weights are shifted by the embedding of the target token; see Figure 1.

In summary, our contributions are: (i) Analyzing the rank of gradients. (ii) Interpreting gradients by inspecting relatively small spanning sets. In particular, we examine the Vector-Jacobian Product (VJP) obtained during the backward pass, which is the equivalent of the hidden states of the forward pass. (iii) Investigating the embedding of these sets by projecting them into tokens. (iv) Revealing a two-phase mechanism by which models learn to store knowledge in their MLPs, which we termed “imprint and shift”. (v) Exploring a novel editing method based solely on a single forward pass.

## 2 Related Work

Developing methods to explain LMs is central to the interpretability community (Belinkov and Glass, 2019; Srivastava et al., 2023). Initially inspired by interpretability efforts in vision models (Sundararajan et al., 2017; Samek et al., 2017; Zhang and Zhu, 2018; Indolia et al., 2018; Olah et al., 2020), LMs have also benefited from the ability to operate in the language domain. This includes leveraging projections of vectors into readable concepts (nostalgebraist, 2020; Simhi and Markovitch, 2023) or clustering them into the idioms they promote (Cunningham et al., 2023; Tamkin et al., 2023; Tigges et al., 2023; Bricken et al., 2023).

Reverse engineering the gradient’s role in shifting model behavior has been a primary method to comprehend the mechanics of deep learning models. Recent work (Ilharco et al., 2022; Gueta et al., 2023; Tian et al., 2023) demonstrates that clustering the weights that models learn during training or fine-tuning reveals patterns that connect the tasks and their training data. Other approaches employing Saliency Maps (Simonyan et al., 2014) explore the relationship between a gradient matrix and parts of its corresponding forward pass input, where we have observed that gradients are spans, linear combinations, of those inputs. Our work differs from previous studies in two key aspects: (1) we interpret gradients based on the components that generate them, primarily the VJPs, rather than analyzing

entire gradient matrices, and (2) we elucidate the information stored in the gradients and project them into tokens, instead of solely examining their impact on the model’s predictions or their connection to the training data. To the best of our knowledge, we are the first to explore both projecting gradients into tokens and examining them through VJPs, offering a perspective that has yet to be explored.

Our experiment regarding LM editing adds to the line of work that utilizes interpretability for knowledge editing. The closest idea to our implementation was introduced by Dai et al. (2022), who identified activated neurons for specific idioms in encoder LMs and altered them by injecting the embedded target. We show that gradients work in a very similar way. Other state-of-the-art model editing methods include Mitchell et al. (2021) and Meng et al. (2022, 2023).

Our work analyzes the gradient’s rank. It was previously known that gradients are low-rank (Mitchell et al., 2021), but the utilization of this characteristic for interpretability study or predicting the rank of an edited prompt have remained unexplored. Optimizers and adaptors, such as LoRA (Hu et al., 2022), were created to constrain the rank of gradients, making fine-tuning faster. In contrast, our work demonstrates that the ranks of gradients can be predicted based on the edited prompt length.

## 3 Background

We first provide background on transformers, focusing on the components that play a role in our analysis and omitting other concepts, such as Layer Norms and positional embedding, which are explored in full by Vaswani et al. (2017) and Radford et al. (2019). We then provide the necessary background on the backward pass; a more comprehensive description is given by Clark (2017) and Bishop (2006). Finally, we discuss the building blocks of the Logit Lens method.

### 3.1 Transformer LMs

Generative Pre-trained Transformer (GPT), is an auto-regressive family of architectures containing multiple transformer blocks. Given a prompt sequence of  $n$  tokens, GPT predicts a single token. The architecture maintains an embedding dimension  $d$  throughout all layers. First, the input tokens are embedded using an embedding matrix  $E$  into vectors  $X = [x_1, \dots, x_n] \in \mathbb{R}^{n \times d}$ . This is mirrored at the final stage, in which a decoding matrix

$D$  projects the output of the last transformer block into a score for each token within the vocabulary.

Each transformer block comprises an attention layer (Attn) and a Multi-Layer Perceptron (MLP) layer, interconnected by a residual stream. The attention mechanism transfer vectors (information) from each of the preceding inputs to the current forward pass. In our study, we do not delve into this module and refer the reader to [Radford et al. \(2018\)](#) for more details.

The MLP layer (also known as FFN, Feed-Forward Network) consists of two fully connected matrices  $FF_1, FF_2^T \in \mathbb{R}^{d \times d_m}$ , with an activation function  $f$  between them:  $\text{MLP}(X) = f(XFF_1)FF_2$ .

Hence, the calculation that the  $l$ -th transformer block performs on its input hidden state,  $X^l$ , is given by  $X^{l+1} = X^l + \text{Attn}(X^l) + \text{MLP}(\text{Attn}(X^l) + X^l)$ .

### 3.2 Backpropagation

Backpropagation ([Rumelhart et al., 1986](#); [Le Cun, 1988](#)) is an application of the chain rule to compute derivatives and update weights in the optimization of deep learning network-based models. The process begins with the model executing a forward pass, generating a prediction  $\hat{y}$ , which is subsequently compared to a desired target by quantifying the disparity through a loss score  $L$ . Following this, a backward pass is initiated, iterating through the model’s layers and computing the layers’ gradients in the reverse order of the forward pass.

For a given layer of the model that during the forward pass computed  $z = xW$ , where  $x \in \mathbb{R}^{d_1}, z \in \mathbb{R}^{d_2}$  are its intermediate input and output, we compute its gradient matrix using the chain rule:

$$\frac{\partial L}{\partial W} = \frac{\partial z}{\partial W} \frac{\partial L}{\partial z} \in \mathbb{R}^{d_2 \times d_1} \quad (1)$$

We can directly compute  $\frac{\partial z}{\partial W} = \frac{\partial xW}{\partial W} = x^\top$ . The other derivative  $\delta = \frac{\partial L}{\partial z} \in \mathbb{R}^{d_2}$  is known as the Vector-Jacobian Product (VJP) of  $z$ . It can be thought of as the hidden state of the backward pass and is the error factor that later layers project back.

In LMs, the output of the model is an unnormalized vector,  $\hat{y} \in \mathbb{R}^{|\text{vocabulary}|}$ , representing a score for each of the model’s tokens. We denote the target token by an index  $t \in [|\text{vocabulary}|]$ . Typically the Negative Log-Likelihood (NLL) loss is used:

$$\hat{p} = \text{Softmax}(\hat{y}) \in \mathbb{R}^{|\text{vocabulary}|} \quad (2)$$

$$L = \text{NLL}(\hat{p}, t) = -\log(\hat{p}[t]) \in \mathbb{R} \quad (3)$$

where  $\hat{p}$  represents the normalized probabilities of  $\hat{y}$  and  $[t]$  is its  $t$ -th value (the target token’s probability). For the last layer’s output  $z = \hat{y}$ , calculating its  $\delta$  (VJP) can be done directly by ( $k \in [|\text{vocabulary}|]$ ):

$$\delta[k] = \begin{cases} \hat{p}[k] - 1 \leq 0 & \text{if } k = t \\ \hat{p}[k] \geq 0 & \text{otherwise} \end{cases} \quad (4)$$

For an earlier layer in the model  $l$ , we cannot compute the VJP of its output  $z^l$  directly (here  $l$  indicates the layer’s index). Since we iterate the model in a reverse order, we can assume we already computed the VJP of layer  $l + 1$ . If the layers are sequential, the output of layer  $l$  is the input of  $l + 1$ , therefrom  $z^l = x^{l+1}$ . Utilizing the backward step, we can compute:

$$\delta^l = \frac{\partial L}{\partial z^l} = \frac{\partial L}{\partial x^{l+1}} = \delta^{l+1}(W^{l+1})^\top \quad (5)$$

To summarize, in deep learning models, the gradient of a loss function  $L$  with respect to a given layer  $W$ , is the outer product of the layer’s forward pass input,  $x$ , and its output  $z$ ’s VJP,  $\delta$ :

$$\frac{\partial L}{\partial W} = \frac{\partial z}{\partial W} \frac{\partial L}{\partial z} = x^\top \cdot \delta \in \mathbb{R}^{d_2 \times d_1} \quad (6)$$

### 3.3 Vocabulary Projection Methods

[nostalgebraist \(2020\)](#) discovered that we can transform hidden states from LMs forward passes into vocabulary probabilities, thereby reflecting their intermediate predictions. Termed as **Logit Lens (LL)**, this method projects a vector  $x$  in the size of the embedding space  $d$  by applying it with the LM’s decoding, the process that transforms the last transformer block’s output into a prediction:

$$\text{LL}(x) = \text{Softmax}(\ln_f(x)D) \in \mathbb{R}^{|\text{vocabulary}|} \quad (7)$$

where  $\ln_f$  is the model’s last Layer Norm before the decoding matrix  $D$ .

The projection captures the gradual building of LMs output ([Millidge and Black, 2022](#); [Haviv et al., 2023](#)), and projections from later layers are more interpretable than earlier ones. Efforts such as [Din et al. \(2023\)](#) and [Belrose et al. \(2023\)](#) try to solve this gap by incorporating learned transformations into LL. However, to emphasize our main discoveries, we have not included such enhancements, which primarily aim to shortcut the models’ computations and require dedicated training procedures.

An artificial neuron performs a weighted sum of its inputs, and appears as a column or a row of